

ilsh — the virtual filesystem

`ilsh` can present a small Unix-like directory tree over the real Windows filesystem. It is **off by default**, so scripts written before this feature behave exactly as they did (real Windows paths everywhere). Turn it on with a command-line flag or from your startup script.

Turning it on

```
ilsh --home C:\Users\me\project      # enable; /home points at that directory
```

or, inside the shell (e.g. from `.ilshellrc`):

```
vfs on          # enable; /home = the current real directory
vfs on C:\path  # enable; /home = C:\path
vfs off         # back to plain real-path mode
vfs status      # show the current layout
```

The layout

Virtual	Default real target	Notes
<code>/</code>	— (virtual)	<code>ls /</code> lists the mounts
<code>/home</code>	the <code>--home</code> directory	your working directory; the prompt starts here
<code>/home/windows</code>	<code>%USERPROFILE%</code> (e.g. <code>C:\Users\me</code>)	your real Windows home
<code>/bin</code>	<code><repo>\out</code>	the compilers and utilities
<code>/etc</code>	<code><repo>\etc</code>	config files
<code>/include</code> (or <code>/includes</code>)	<code><repo>\include</code>	include files
<code>/lib</code>	<code><repo>\out</code>	.NET libraries the compilers need
<code>/tmp</code>	<code><repo>\tmp</code>	scratch space

Each mount is just a **shell variable** holding a real Windows path (`home` , `bin` , `etc` , `include` , `lib` , `tmp` , `windows`). Repoint any of them at any time — and several virtual folders may point at the same real folder:

```
bin=C:\tools\bin
include=C:\myproject\include
etc=C:\myproject          # /etc and /home can share a real folder
```

Reaching the real filesystem

When the VFS is on, ordinary commands (`cd`, `pwd`, `ls`, `cat`, `cp`, redirections, ...) work in virtual space. Three companions reach the **real** Windows filesystem directly:

```
lcd C:\Windows\System32      # change the real working directory
lpwd                         # print the real working directory
lls -al C:\Users\me          # list a real directory
```

Startup script

An interactive shell runs `~/.ilshellrc` once at startup, from `/home`. Like any dotfile it is hidden from `ls` unless you pass `ls -al`. It is the preferred place to configure the shell — enable the VFS, repoint mounts, define aliases. If there is no `.ilshellrc`, `ilsh` falls back to `~/.bashrc`. A template lives in `shell/dot-ilshellrc.sample`.

Built-in tools

Beyond the file/text coreutils, `ilsh` has these built in (type `NAME -h` for usage):

Command	What it does
<code>bc [expr]</code>	scientific calculator (REPL with no expression)
<code>date [+FORMAT]</code>	current date/time; <code>FORMAT</code> uses <code>%Y %m %d %H %M %S %A %B %a %b %j %p %y</code>
<code>time CMD ...</code>	run a command and report wall-clock time
<code>sed [-n] SCRIPT [file]</code>	stream editor: <code>s/old/new/[g][p]</code> , <code>/pat/d</code> , <code>/pat/p</code> (literal match)
<code>wc [file...]</code>	line / word / byte counts
<code>man NAME</code>	page a language's reference doc (<code>man pascal</code> , <code>man lua</code> , <code>man shell</code>) through <code>more</code>
<code>ps [-e]</code>	list shell-started background jobs; <code>-e</code> also dumps the OS <code>tasklist</code>
<code>vi FILE</code>	modal editor with syntax highlighting (<code>:syntax on</code> / <code>:syntax off</code>)

`vi` colors keywords, strings, comments, numbers, and (for C) preprocessor lines, with a **language profile chosen by file extension** — comment and string syntax follow the language, not just the keyword set:

Language	Extensions	Line comment	Block comment
C / C++	<code>.c .h .cpp .cc .y .l</code> (default)	<code>//</code>	<code>/* */</code>
Pascal	<code>.pas .pp</code>	<code>//</code>	<code>{ }</code> and <code>(* *)</code>
Lua	<code>.lua</code>	<code>--</code>	<code>--[[]]</code>
Lisp/Scheme	<code>.lisp .lsp .scm .el</code>	<code>;</code>	<code>#\ \ #</code>
Shell	<code>.sh</code>	<code>#</code>	—
Fortran	<code>.f90 .f .f95</code>	<code>!</code>	—
Ada	<code>.adb .ads</code>	<code>--</code>	—
Prolog	<code>.pl .pro</code>	<code>%</code>	<code>/* */</code>
BASIC	<code>.bas</code>	<code>'</code>	—

Keywords are matched case-insensitively for Pascal/Ada/Fortran/BASIC. Block comments are tracked across lines. Highlighting is on by default; `:syntax off` disables it and `:syntax on` re-enables it.

The windowed terminal (ilterm)

`ilterm` is an Avalonia window that hosts the shell with:

- **16-color output** — the shell colors its prompt (cyan) and directory names in `ls` (bright blue); any program can emit standard ANSI SGR colors (`\x1b[31m` ...).
- **Right-click menu** — Copy, Paste, Cut, **New shell window** (launches a duplicate, forwarding the same `--home`), and a **Font** submenu (larger/smaller, and a choice of monospace families).
- **Selection** — drag with the mouse to select; Copy (or Ctrl+C) puts it on the clipboard; Paste (or Ctrl+V) types the clipboard into the shell.
- **.quicklaunch** — a file in your home directory whose `Label = command` lines become extra menu entries that type the command and run it. Template in `shell/dot-quicklaunch.sample`.

Backward compatibility

With the VFS off (the default), `vmap()` is the identity function: every path passes through untouched and the shell behaves exactly as before. The feature only changes behavior once you opt in.